AP9EV – Genetický algoritmus, reálné parametry

František Hromek
Model: Claude Opus 5.5

Fáze 1 – Pochopení a testovací funkce

> Navazuju na minulý úkol, kde mám hotový binární genetický algoritmus v ga.py. Teď ho potřebuju použít na optimalizaci reálných funkcí. Vysvětli mi krátce, v čem je to jiné než OneMax a proč vůbec musím řešit ‘reprezentaci’ čísla — zatím bez kódu. Pak mi napiš tři testovací funkce: Sphere, Rosenbrock a Schwefel pro 10D, a ke každé připiš rozsah (bounds) a kde má globální minimum.

U OneMaxu byl bitový řetězec přímo řešením. Teď je řešením vektor deseti reálných čísel, takže mezi bity a bodem, do kterého dosazujeme, musí být převod – reprezentace. Ta rozhoduje, **jak velkou změnu hodnoty způsobí mutace jednoho bitu**, jakou přesnost a rozsah lze vyjádřit a kolik bitových řetězců je vůbec platných. Stejný GA se proto v různých reprezentacích chová úplně jinak. Real-valued GA převod vynechá a mutuje přímo čísla.

Funkce jsou v `functions.py`, všechny se minimalizují a mají minimum f = 0:

Funkce	Rozsah	Minimum	Charakter
Sphere	[−5.12, 5.12]	x_i = 0	hladká, unimodální – kontrola, že GA funguje
Rosenbrock	[−5, 10]	x_i = 1	úzké zahnuté údolí, proměnné jsou provázané
Schwefel	[−500, 500]	x_i = 420.97	~8 lokálních minim na dimenzi, optimum u kraje

Fáze 2 – Tři binární reprezentace

> První reprezentace, IEEE 754: převeď reálné číslo na 32bitový řetězec a zpátky (float32 = znaménko, exponent, mantisa). Napiš encode/decode, vysvětli mi roli exponentu a mantisy a proč může přehození jednoho bitu v exponentu úplně rozhodit hodnotu. Vyzkoušej to na pár číslech tam a zpět.

float32 = 1 bit znaménka, 8 bitů exponentu, 23 bitů mantisy; hodnota = ±1.mantisa · $2^{\text{exponent}-127}$. **Exponent** určuje řád čísla, **mantisa** polohu uvnitř řádu. Bit exponentu proto hodnotu násobí mocninou dvojky – z 3.14 se jedním bitem stane 6.28, 205 887 nebo 6.7e-39. Encode/decode je jen reinterpretace float32 jako uint32 (`representations.py`, třída `IEEE754`). Tam a zpět projdou 0, 1, −2.5 přesně, 3.14159 s chybou 1e-7 a 420.9687 s chybou 1e-5 (relativní přesnost 23bitové mantisy).

> Druhá reprezentace, fixed-point na 32 bitech — pevný počet bitů na celou a na desetinnou část, aby pokryl bounds funkce. Napiš encode/decode a vysvětli mi, jak volím poměr bitů a čím je to omezené.

Celé číslo ve dvojkovém doplňku dělené $2^{\text{frac_bits}}$. Celá část se volí co nejmenší, aby pokryla největší |mez|: `int_bits = ⌊log2(max|mez|)⌋ + 1`, zbytek jde do desetinné části. Sphere tak má 1 + 3 + 28 bitů (krok 4e-9), Rosenbrock 1 + 4 + 27, Schwefel 1 + 9 + 22 (krok 2e-7). **Omezení:** 32 bitů je pevný rozpočet – bit navíc pro rozsah znamená poloviční přesnost, krok je všude stejný a kódovaný rozsah je mocnina dvojky, takže přesahuje meze (u Sphere ±8 místo ±5.12).

> Třetí reprezentace, BCD — 4 bity na každý desítkový řád, celkem 32 bitů. Napiš encode/decode a vysvětli mi, proč je ‘plýtvavé’ (6 z 16 kombinací 4 bitů nic neznamená). Pak mi do pár vět shrň, co udělá mutace jednoho bitu u každé z těch tří reprezentací — tím budu zdůvodňovat výsledky.

8 nibblů = znaménko + 7 desítkových číslic, počet číslic před čárkou podle mezí (Sphere 1 + 6, Rosenbrock 2 + 5, Schwefel 3 + 4). Nibbly 10–15 nic neznamenají a ořezávají se na 9. Sedm číslic dává jen 10^7 hodnot místo 2^{28} – využije se **3.7 % kombinací**. Navíc nejde přizpůsobit rozsah: Rosenbrock kvůli mezi 10 potřebuje dvě číslice, kóduje tedy ±99.99 a náhodný řetězec padne do mezí jen ve **4 %** případů.

Mutace jednoho bitu u x = 3.14159 (výstup `python representations.py`, meze Schwefela):

	Velký skok	Střední	Jemný
IEEE 754	exponent: 6.7e-39 / 205 887	nejnižší bit exp.: 6.28 (×2)	mantisa: 3.1411
Fixed-point	znaménko: −508.9; 2^8: 259.1	2^0: 2.14	2^{-11}: 3.1411
BCD	stovky: 803.1	jednotky: 9.14 (neplatná → 9)	setiny: 3.1916

- **IEEE 754** mění hodnotu **multiplikativně**: kolem nuly ladí extrémně jemně, ale bit exponentu pošle hodnotu o řády jinam nebo za meze.
- **Fixed-point** mění hodnotu **aditivně a předvídatelně**: bit k přičte vždy stejnou mocninu dvojky, od hrubých kroků po jemné.
- **BCD** mění hodnotu **aditivně, ale nepravidelně**: o 1–8 jednotek desítkového řádu, často vznikne neplatná číslice a mnoho mutací nic nezmění.

Fáze 3 – Napojení na GA a real-valued varianta

Napoj reprezentace na můj binární GA z ga.py: jedinec = 10×32 = 320 bitů, před výpočtem fitness se rozkóduje po 32 bitech na 10 čísel a dosadí do funkce. Je to minimalizace — vysvětli mi, jak ji v GA udělat (třeba -f nebo 1/(1+f)). A ošetři, že IEEE 754 může vrátit hodnotu mimo rozsah nebo NaN/inf (clamping do bounds), ať to nespadne.

`run_ga` z minulého úkolu zůstává beze změny, mění se jen fitness (`ga_real.py`):

```
def fitness(bits):
    return -f(decode_in_bounds(rep, bits, f))   # 320 bitů -> 10 čísel -> -f(x)

def decode_in_bounds(rep, bits, f):
    x = rep.decode(bits)
    x = np.where(np.isnan(x), np.copysign(np.inf, x), x)   # NaN jako ±inf
    return np.clip(x, f.lower, f.upper)                    # mimo meze -> na mez
```

Minimalizace: fitness = −f. Pořadová selekce bere jen pořadí jedinců, takže jí záporná fitness nevadí. 1/(1+f) by šlo s ruletou, ale u malých f mají všichni fitness ≈ 1 a selekční tlak zmizí. **NaN/inf** se berou jako nekonečno se svým znaménkem a ořežou se na mez – nic nespadne, jedinec jen dostane špatnou fitness. U IEEE 754 je to časté: asi polovina náhodných hodnot má |x| ≥ 512.

Teď druhý přístup — real-valued GA, kde jedinec je rovnou pole 10 reálných čísel bez kódování. Napiš jednobodové křížení mezi dimenzemi a dvě varianty mutace: (a) x + N(0, sigma²), (b) absolutně náhodná hodnota z celého rozsahu. Vysvětli mi rozdíl v chování těch dvou mutací.

Křížení je `one_point_crossover` z `ga.py` – na poli 10 čísel řeže přesně mezi dimenzemi. Selekce a elitismus jsou taky převzaté, varianty se tedy liší jen mutací (každá souřadnice s pravděpodobností p_mut):

```
x + rng.normal(0, sigma)            # (a) normální – ořízne se do mezí
rng.uniform(f.lower, f.upper)       # (b) náhodná – nezávisí na původní hodnotě
```

Normální mutace hledá **lokálně** – umí doladit, ale jen v měřítku σ, a do vzdálené oblasti (jiné minimum Schwefela) se nedostane. **Náhodná mutace** hledá **globálně** – skočí kamkoli, ale nic nedoladí; okolí optima trefí jen náhodou.

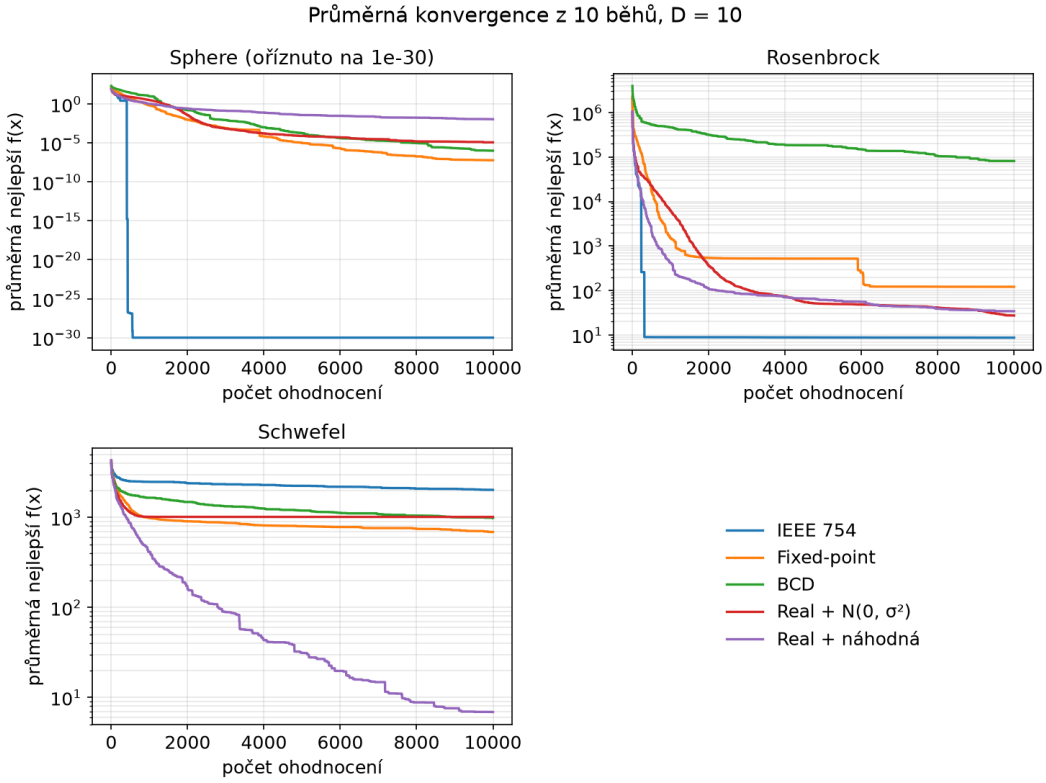
Fáze 4 – Experimenty, grafy, závěr

Spoj to do experimentu: 3 funkce × 5 variant (IEEE754, fixed-point, BCD, real-valued s normální mutací, real-valued s náhodnou mutací), vše 10D, ukončení po 10000 ohodnoceních, 10 běhů. Ulož průměrné konvergenční křivky a finální statistiky (nejlepší, nejhorší, průměr, medián, směrodatná odchylka). Vykresli grafy (X = počet ohodnocení, Y = nejlepší hodnota) a vypiš statistiky do tabulky. Napiš, jak to spustit jednou funkcí.

Vše se spustí příkazem `python GeneticAlgorithmRealParametres/experiment.py` (asi 50 s), z Pythonu `experiment()` a `compare_mutation()`. Nastavení z minulého úkolu: populace 20, elitismus 20 %, rank selekce; binární varianty p_mut = 1 % na bit, real-valued p_mut = 0.1 na souřadnici a σ = 1 %

rozsahu. Plné statistiky (nejlepší, nejhorší, průměr, medián, sm. odchylka) jsou ve `stats.md`, zde průměr finálního f (tučně nejlepší):

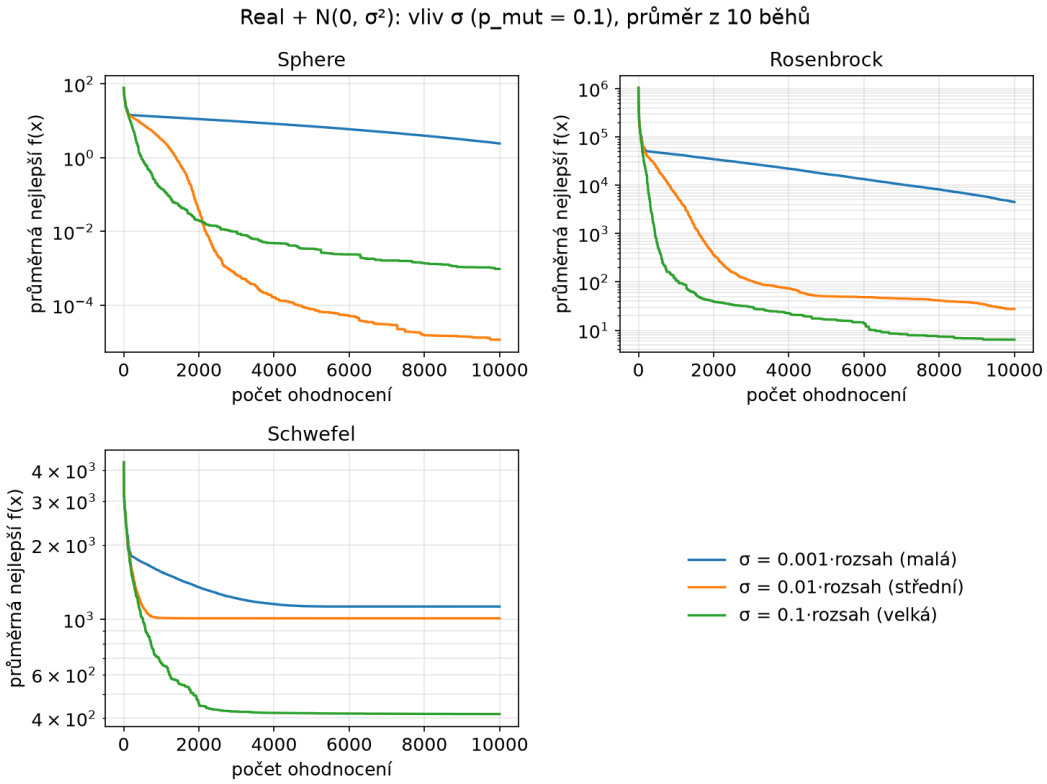
Varianta	Sphere	Rosenbrock	Schwefel
IEEE 754	**2.4e-81**	**8.74**	2 020
Fixed-point	6.0e-08	121	688
BCD	1.0e-06	81 290	986
Real + N(0, σ²)	1.2e-05	27.5	1 011
Real + náhodná	0.0111	34.3	**6.88**



Obr. 1 – Průměrná konvergence z 10 běhů (log. osa; Sphere oříznuto na 1e-30, IEEE 754 končí na ≈ 1e-81).

- **Sphere:** vyhrává IEEE 754 – minimum je v nule a tam má float32 nejhustší mřížku.
- **Rosenbrock:** IEEE 754 má nejlepší průměr, ale jen proto, že ve všech bězích uvízne u nuly (f ≈ 9); k optimu v jedničkách se nedostane. BCD selhává – 96 % hodnot se ořízne na mez. Fixed-point kazí průměr jeden běh (826), jeho medián je 12.
- **Schwefel:** vyhrává náhodná mutace – funkce je separabilní a náhodné přegenerování souřadnice spolu s křížením mezi dimenzemi prohledá každou osu zvlášť. IEEE 754 přilepí ořezané souřadnice na −500.

U real-valued s normální mutací zkus 2–3 hodnoty sigma (malá/střední/velká) a ukaž, jak to ovlivní konvergenci. Pak mi pomoz to zhodnotit vlastními slovy: která reprezentace/varianta byla na kterých funkcích nejlepší a proč — ať to souvisí s tím, jak mutace mění hodnotu u jednotlivých reprezentací. Dej mi 4–5 vět jako závěr do protokolu.



Obr. 2 – Vliv σ normální mutace, p_mut = 0.1 (průměr finálního f: Sphere 2.4 / 1.2e-5 / 9.7e-4, Rosenbrock 4 511 / 27.5 / 6.4, Schwefel 1 128 / 1 011 / 416).

Malá σ (0.001·rozsah) nestihne z náhodného startu dojít nikam. Střední σ vyhrává na Sphere, kde je potřeba doladit. Velká σ (0.1·rozsah) vyhrává na Rosenbrocku a Schwefelovi, kde je potřeba přeskakovat – s ní by normální mutace vyhrála i Rosenbrocka (6.4 proti 8.7 u IEEE 754), hlavní srovnání ji se σ = 1 % tedy znevýhodňuje. Pevná σ nikdy nesedí všude; lepší by byla σ, která se během běhu zmenšuje. Vliv p_mut (0.05 / 0.1 / 0.3) je menší a je v `mutation_comparison.md` a `p_mut_comparison.png`.

Závěr

O výsledku nerozhoduje samotný GA, ale to, jak velký krok v prostoru x udělá mutace v dané reprezentaci, a jestli sedí na tvar funkce. IEEE 754 vyhrál na Sphere (1e-81), protože jeho mutace je multiplikativní a mřížka je nejhustší kolem nuly, stejná vlastnost ho ale na Rosenbrocku stáhla k nule místo k optimu a na Schwefelovi přilepila ořezané souřadnice na mez. Fixed-point se díky pevným krokům bitů od hrubých po jemné choval nejvyrovnaněji (druhý na Sphere i Schwefelovi), zatímco BCD bylo nejhorší, protože plýtvá kombinacemi bitů a jeho rozsah nejde přizpůsobit mezím. Normální mutace ladí jen v měřítku σ – střední σ byla nejlepší na Sphere, velká na Rosenbrocku a Schwefelovi. Absolutně náhodná mutace nic nedoladí, ale na multimodálním a separabilním Schwefelovi s velkým náskokem vyhrála (6.9 proti stovkám), protože spolu s křížením mezi dimenzemi prohledává každou souřadnici po celé ose.